Q-learning for Taxi-v3

Topics in AI | Assignment 1 | 20 October 2026

Areej Suleman | COMSATS University Islamabad, Attock Campus

1. Problem description

The task is to learn a policy that picks up a passenger and delivers them to a designated destination while avoiding unnecessary moves and illegal actions. Taxi-v3 is a deterministic 5-by-5 grid environment with internal walls and four named locations. Initial states vary across episodes. Gymnasium supplies the simulator; the learning algorithm is implemented from scratch with NumPy.

Component	Representation
State	(taxi row, taxi column, passenger location, destination). The declared space has 5 x 5 x 5 x 4 = 500 integer states.
Actions	0 south, 1 north, 2 east, 3 west, 4 pick up, 5 drop off.
Rewards	-1 for an ordinary step; -10 for an illegal pickup/drop-off; +20 for successful delivery. These rewards replace one another on a step.
End condition	Successful delivery terminates an episode. The wrapper truncates at 200 steps.

2. Methodology

The agent stores a 500 x 6 Q-table, initialized to zero. During training, epsilon-greedy selection chooses a uniformly random action with probability epsilon; otherwise it chooses an action with maximal Q-value, breaking ties randomly. No action mask or transition model is used.

Update: $Q(s,a) \leftarrow Q(s,a) + \alpha [y - Q(s,a)]$
Target: $y = r$ for a terminal transition; otherwise $y = r + \gamma \max_{a'} Q(s',a')$.

This is off-policy Q-learning: the exploratory behavior policy collects transitions, while the target uses the greedy next-state value. Truncation stops an episode but retains the bootstrap term, since a time limit does not make the underlying state terminal.

Architecture: agent.py implements action selection and the update; utils.py implements rollouts, evaluation, plotting and GIF rendering; main.py coordinates experiments and saves outputs. The notebook executes the same pipeline and displays results.

3. Experimental setup

Gymnasium is pinned to 1.2.3 to reproduce the requested Taxi-v3 environment. Each learning rate (0.01, 0.1, 0.5) is trained for 10,000 episodes using seeds 42, 43 and 44: nine runs in total. The reset seed for training episode i is training_seed + i. Agent exploration uses its own seeded NumPy generator.

Every 1,000 episodes, a separate environment measures greedy performance on 100 fixed reset seeds (50000-50099). We select the learning rate with the largest mean validation return across the first three checkpoints and all three training seeds; this rewards faster early learning. We retain the seed-42 model for that learning rate, rather than selecting the best seed. The final test uses 300 reset seeds (10000-10299), with no training or exploration. The random baseline uses the same test reset seeds and samples all six actions uniformly.

Parameter	Value
Environment / algorithm	Taxi-v3 / tabular Q-learning
Selected learning rate (alpha)	0.5
Discount factor (gamma)	0.99
Training budget	10,000 episodes per run; 3 seeds per learning rate
Exploration schedule	epsilon(i) = max(0.05, 0.9995^i), i starts at 0
Episode limit / Q initialization	200 steps / all zeros
Validation / final test	100 episodes per checkpoint / 300 final episodes
Evaluation policy	Greedy argmax, epsilon = 0, no Q updates

4. Hyperparameter sensitivity

Alpha	Early validation return: mean +/- SD	Final validation success
0.01	-197.86 +/- 1.23	52.7%
0.1	-33.74 +/- 4.29	100.0%
0.5	7.93 +/- 0.39	100.0%

Early return averages checkpoints at 1,000, 2,000 and 3,000 episodes within each run; SD is the sample standard deviation across the three run-level averages. Final success is averaged across runs at 10,000 episodes. Only alpha is varied; conclusions do not establish sensitivity to gamma or epsilon decay.

5. Results and discussion

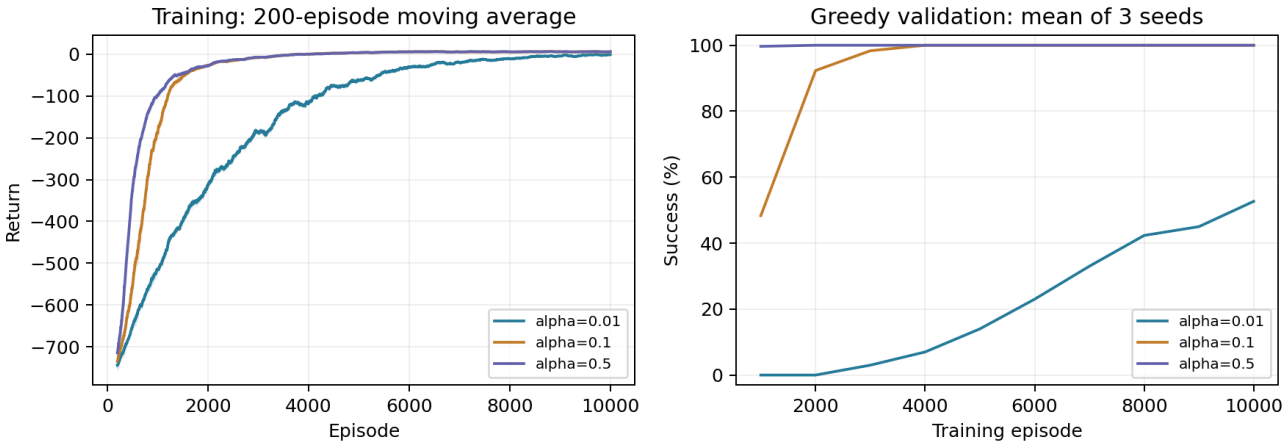


Figure 1. Left: mean 200-episode smoothed exploratory training return, with min-to-max shading across three seeds. Right: mean greedy validation success across the same runs.

Test policy	Success rate	Mean return	Mean steps
Uniform random	3.7%	-764.49	196.82
Q-learning	100.0%	7.97	13.03

Table 2. Results from 300 final episodes per policy. Mean steps includes unsuccessful episodes that reach the time limit. Success requires terminal delivery with reward +20; evaluation makes no learning updates.

The selected alpha=0.5 model achieves 100.0% test success versus 3.7% for random actions. Its mean return is 7.97. Faster successful routes receive larger returns because each ordinary step costs one reward. Higher early validation return favors the selected learning rate under the fixed exploration schedule.

The learning curves distinguish exploration from policy quality: training can retain negative spikes because epsilon never falls below 0.05, even when greedy validation succeeds. A low learning rate propagates useful values slowly; illegal pickup/drop-off actions and repeated movement into walls hurt early learning. The random policy frequently wastes steps and incurs illegal-action penalties.

The saved GIF uses reset seed 2026 and shows successful delivery in 7 steps with return 14. It illustrates one episode, while the test table gives broader evidence.

Limitations: only three training seeds and one environment were studied. Some final test reset seeds overlap training reset seeds, and different seeds may produce repeated states in this small finite problem; they are not 300 distinct scenarios. Evaluation tests familiar environment dynamics, not transfer to unseen maps. Empirical stabilization is not a proof of optimality or theoretical convergence, and no real-world driving claim is made.

6. Conclusion

A small Q-table is sufficient to learn effective passenger pickup and delivery in Taxi-v3. The measured comparison shows the importance of learning rate for early learning. Separate validation and final testing, a saved model and an executed notebook make the experiment reproducible.

References

Farama Foundation. Taxi, Gymnasium 1.2.3 documentation.
https://gymnasium.farama.org/v1.2.3/environments/toy_text/taxi/
Sutton, R. S. and Barto, A. G. Reinforcement Learning: An Introduction, 2nd ed., 2018, Chapter 6 (Q-learning).